

《深度学习平台与应用》作业四 (20241221)

1.损失函数计算

已知一个线性分类器的预测得分为以下矩阵:

$$S = \begin{bmatrix} 4.8 & 2.5 & 2.0 \\ -3.1 & 0 & 12.9 \end{bmatrix}$$

a) 其中每一行表示一个样本的预测得分, 每一列表示一个类别。假设正确类别的索引为[0,2],请计算SVM的损失函数值, 公式如下:

$$L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

b) 对于这个线性分类器的预测, 计算出softmax概率以及交叉熵损失。

2.numpy 实现带L2正则化的逻辑回归

```
import numpy as np
import matplotlib.pyplot as plt

# 生成高斯分布的二分类数据
def generate_data(n_samples=100, mean1=(2, 2), mean2=(-2, -2), cov=((1, 0), (0, 1))):
    np.random.seed(42)
    X1 = np.random.multivariate_normal(mean1, cov, n_samples) # 正类
    X2 = np.random.multivariate_normal(mean2, cov, n_samples) # 负类
    Y1 = np.ones((n_samples, 1))
    Y2 = np.zeros((n_samples, 1))
    X = np.vstack((X1, X2))
    Y = np.vstack((Y1, Y2))
    indices = np.arange(X.shape[0])
    np.random.shuffle(indices)
    return X[indices], Y[indices]

# Sigmoid 函数
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# 逻辑回归 (带 L2 正则化)
def logistic_regression(X, Y, lr=0.01, epochs=1000, lambda_=0.1):
    m, n = X.shape # 样本数量和特征数量
    w = np.zeros((n, 1)) # 初始化权重
    b = 0 # 初始偏置
    losses = []

    for epoch in range(epochs):
        # 前向传播
        z = np.dot(X, w) + b # 线性部分
```

```

    A = sigmoid(Z) # Sigmoid 激活
    #todo 损失函数（交叉熵 + L2 正则化项）
    #loss = ?
    losses.append(loss)

    #todo 反向传播
    #dw = ? 交叉熵 + L2 正则化 梯度
    #db = ? 偏置的梯度

    # 参数更新
    w -= lr * dw
    b -= lr * db

    return w, b, losses

# 预测函数
def predict(X, w, b):
    Z = np.dot(X, w) + b
    A = sigmoid(Z) # 激活函数
    return (A >= 0.5).astype(int)

# 绘制数据分布和决策边界
def plot_decision_boundary(X, Y, w, b):
    plt.figure(figsize=(8, 6))
    plt.scatter(X[Y.flatten() == 0][:, 0], X[Y.flatten() == 0][:, 1],
        color='red', label='Class 0')
    plt.scatter(X[Y.flatten() == 1][:, 0], X[Y.flatten() == 1][:, 1],
        color='blue', label='Class 1')
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01), np.arange(y_min, y_max,
0.01))
    grid = np.c_[xx.ravel(), yy.ravel()]
    Z = predict(grid, w, b)
    Z = Z.reshape(xx.shape)
    plt.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
    plt.legend()
    plt.title("Decision Boundary")
    plt.show()

# 主程序
if __name__ == "__main__":
    X, Y = generate_data(n_samples=100)
    X = (X - np.mean(X, axis=0)) / np.std(X, axis=0) # 特征标准化
    w, b, losses = logistic_regression(X, Y, lr=0.1, epochs=1000, lambda_=0.5)
    print(f"最终损失: {losses[-1]}")
    plt.plot(losses)
    plt.title("Loss Curve")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.show()
    # 绘制决策边界
    plot_decision_boundary(X, Y, w, b)

```

3. L1 正则化和 L2 正则化的主要区别是什么？它们分别适用于什么场景？

4. AdaGrad 和 RMSProp 的主要区别是什么？Adam 优化器结合了哪些优化算法的优点？它的主要超参数有哪些？

5. 鞍点和局部最优值的区别是什么？如何通过 Hessian 矩阵的特征值来区分它们？